Caderno de Arquitetura (Architecture Notebook)

Projeto: Compreendendo o ponto

Disciplina: Engenharia de Software

Professor: Fernando Albuquerque Chacon

---

1 Propósito

Este sistema de software provê suporte à gestão ágil de requisitos de software, permitindo o gerenciamento completo de projetos, backlogs de produto e sprint, épicos, histórias de usuário com pontuação RICE, e seus respectivos critérios de aceitação, contando com uma interface condizente com as duas metodologias mais comuns do mercado, Scrum e Kanban.

2 Objetivos Arquiteturais e Filosofia

A arquitetura do sistema pretende seguir normas simples e efetivas de arquitetura de software. Deseja-se implementar um software em duas frentes, back e front, de forma secional e encapsulada, com integridade de dados e de fácil escalabilidade e amnutenibilidade

- **encapsulamento:** Separação entre a camada de apresentação web e a camada de serviços.
- **Integridade de Dados:** Garantia de relacionamentos entre valores diversos e persistência de dados, i.e., RICE, pontos de história e prioridade MSCW
- **Escalabilidade e manutenibilidade:** Utilização de padrões consolidados para permitir fácil navegação e manutenção pelo usuário.

3 Suposições e Dependências

- **Ambiente de Execução:** Assume-se que o usuário final acesse a aplicação por meio de navegadores web modernos, além de que possua a capacidade de efetuar a criação de usuário e ”login” para o uso das funcionalidades do sistema.
- **Ecossistema Node.js / TypeScript:** Dependência da plataforma Node.js para o ecossistema de backend e frontend.
- **Banco de Dados Relacional:** Dependência de um sistema gerenciador de banco de dados relacional leve e acessível localmente.
- **Equipe:** A equipe assume conhecimentos consolidados na linguagem TypeScript e nas ferramentas de controle de versão (Git/GitHub).

4 Requisitos Arquiteturalmente Significativos

- **Autenticação e Sessão de Usuário:** Autenticação prévia para disponibilização dos serviços do sistema.

- **Gestão Hierárquica de Backlogs:** Obrigatoriedade de existência de exatamente um Product Backlog por projeto e suporte a múltiplos Sprint Backlogs para a organização funcional do projeto.
- **Movimentação de Histórias:** Capacidade de transferir e sincronizar histórias de usuário entre backlogs preservando a consistência dos dados.
- **Cálculo Dinâmico RICE:** Execução do cálculo automatizado segundo a fórmula $(Reach \times Impact \times Confidence)/Effort$.

5 Decisões, Restrições e Justificativas

Decisão / Restrição	Justificativa Arquitetural
Adoção de TypeScript no Fullstack	Unificação da linguagem entre cliente e servidor, além de contato prévio da tecnologia com a maioria dos programadores.
Backend Estruturado (NestJS)	Fornece arquitetura modular nativa baseada em Injeção de Dependências, facilitando testes unitários e organização de módulos de domínio.
Mapeamento Objeto-Relacional (Prisma ORM)	Facilita migrações, integridade de dados e consisão de consultas no banco de dados através do sqlite.
Modificação do BD através de serviço	Não haverá acesso direto de componentes de interface ao banco de dados sem a intermediação de controladores e serviços da camada de aplicação.

6 Mecanismos Arquiteturais

6.1 Mecanismo de integridade de dados

Propósito: Gerenciar o ciclo de vida e armazenamento relacional das entidades do sistema (Usuários, Projetos, Backlogs, Épicos, Histórias e Critérios).
Implementação: Banco relacional com controle de migrações e mapeamento com PrismaORM, aplicando integridade de dados.

6.2 Mecanismo de Autenticação

Propósito: Garantir autenticação segura e restringir o acesso a dados apenas ao usuário proprietário do respectivo projeto.
Implementação: verificação de usuário por banco de dados e assimilação para utilização devida da aplicação.

6.3 Mecanismo de Validação e Tratamento de Erros

Propósito: Assegurar consistência nos formatos de entrada de histórias
Implementação: Middleware de validação na camada de entrada e filtros globais de exceção.

7 Abstrações-Chave (Key Abstractions)

- **Usuário (Usuario):** Representa o ator cadastrado e autenticado que possui projetos.

- **Projeto (Projeto):** Unidade agregadora de escopo gerenciada por um usuário.
- **Backlog (Backlog):** Estrutura contenedora de histórias, podendo ser o Product Backlog ou Sprint Backlogs, definido através de um booleano.
- **Épico (Epico):** Agrupamento de histórias de usuário relacionadas a uma funcionalidade de maior magnitude.
- **História de Usuário (HistoriaUsuario):** Unidade fundamental de requisito contendo formato textual padrão, pontuação de esforço, priorização MSCW e métricas RICE.
- **Critério de Aceitação (CriterioAceitacao):** Conjunto de regras verificáveis no padrão Dado/Quando/Então associadas a uma história de usuário.

8 Camadas e Padrão Arquitetural

A arquitetura do sistema adota o modelo em camadas distribuídas:

1. **Camada de Apresentação (Frontend):** Responsável pela renderização de telas, captura de ações do usuário e estética web.
2. **Camada de serviços, controladores e banco de dados (Backend):** Ponto de entrada de requisitos não funcionais e métodos para implantação de serviços.

9 Visões Arquiteturais

9.1 Visão Lógica

Compreende os módulos de software desacoplados por responsabilidade de domínio:

- **Módulo de Usuários:** Gestão cadastral e validação simples.
- **Módulo de Projetos e Backlogs:** Organização de projetos e backlog.
- **Módulo de Histórias, Épicos e Métricas:** Manutenção do ciclo de vida das histórias de usuário e cálculo de pontuações.
- **Modelo Físico de Dados:** Representado pelo esquema de entidades e chaves relacionais implementado no banco de dados.

9.2 Visão Operacional e de Implantação

Descreve os nós físicos e processos de execução do sistema:

- **Cliente (Browser Node):** Executa a interface do usuário localmente via motor do navegador web.

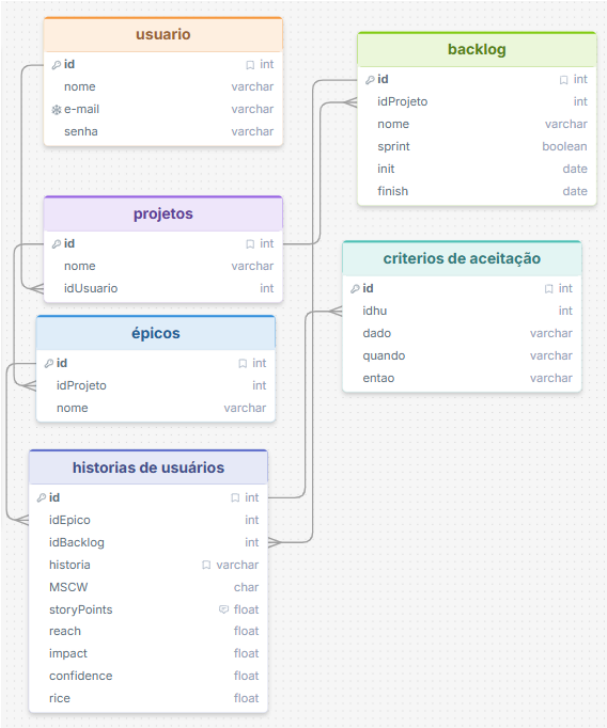


Figure 1: Banco de Dados

- **Servidor de Aplicação (API Node):** Processo de execução que atende chamadas de rede e processa a lógica de domínio.
- **Servidor de Banco de Dados (Database Node):** Serviço dedicado ao armazenamento persistente seguro e indexação dos dados relacionais.

9.3 Visão de Casos de Uso Relevantes

Contempla os fluxos críticos que validam a integração dos mecanismos arquiteturais: autenticação do usuário, criação de projeto e suas dependências (backlogs), vinculação de histórias com critérios de aceitação e reavaliação dinâmica de score RICE.